ОСНОВИ СТВОРЕННЯ RAG-СИСТЕМИ

ПЛАН

1 Особливості роботи та обмеження великих мовних моделей

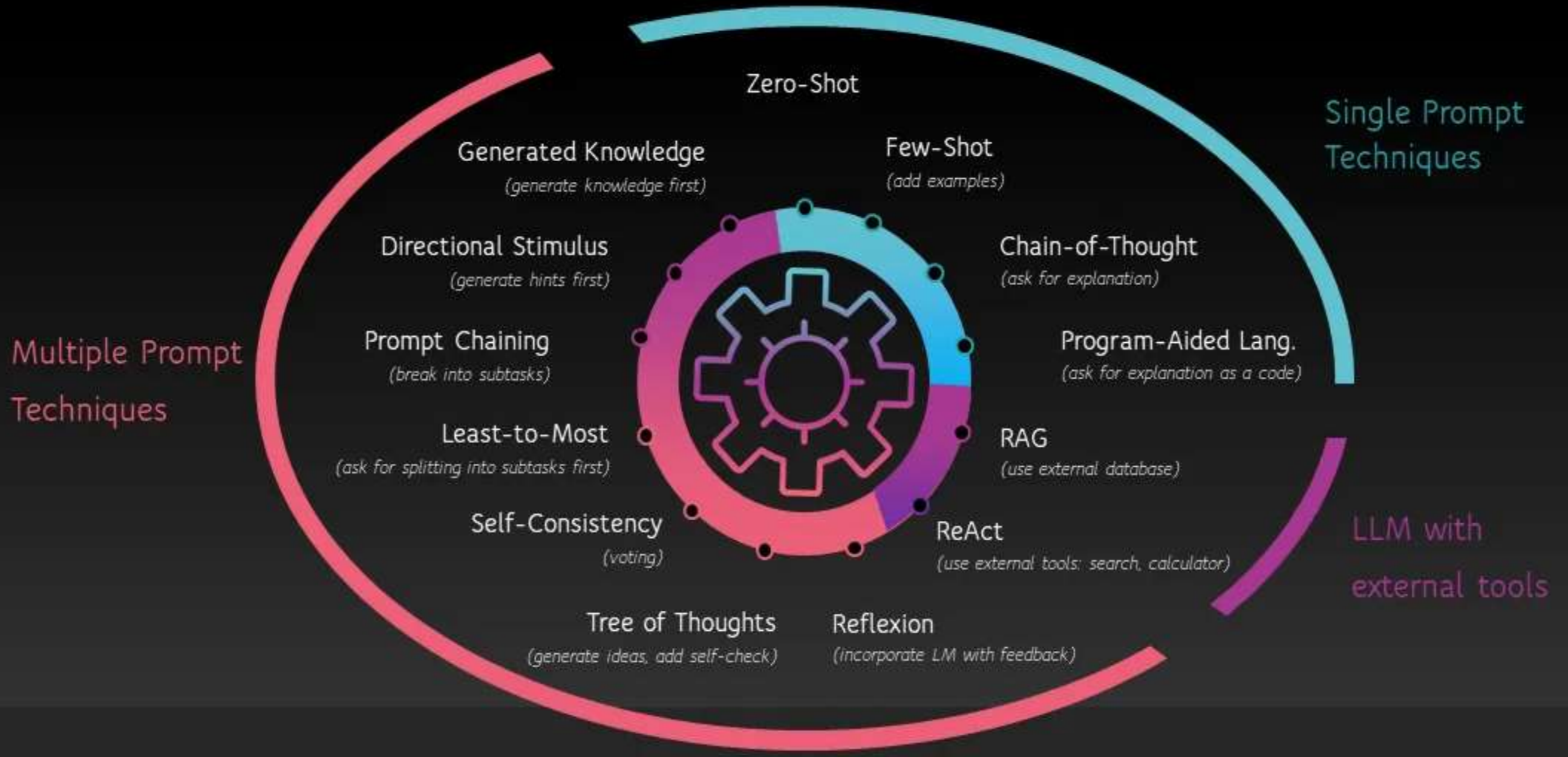
2 Архітектура RAG-систем

3 Основні компоненти

4 Еволюція та способи вдосконалення RAG-систем

ТИПИ ТА ТЕХНІКИ ЗАПИТІВ

Промпт-інжиніринг (Prompt Engineering) – це процес створення запитів до великих мовних моделей з метою отримання точних та релевантних відповідей.



ОБМЕЖЕННЯ ВЕЛИКИХ МОВНИХ МОДЕЛЕЙ

Галюцинації

Впевнене генерування хибних фактів, вигаданих джерел, неіснуючих цитат

Застарілі знання

Модель не знає подій після дати завершення навчання (knowledge cutoff)

Обмеження контекстного вікна

Модель «забуває» початок довгого документа або розмови

Обмеження даних

Нема доступу до корпоративних документів, внутрішніх баз

Чутливість до формулювання запиту

Незначна зміна формулювання суттєво змінює якість відповіді

Проблеми з конфіденційністю

Можливе відтворення персональних даних або захищеного контенту

ПРИКЛАД ПРОБЛЕМИ

1 Загальні питання

Що потрібно для вступу до університету?

Що таке політика повернення товару?

2 Конкретні питання

Які іспити потрібні для вступу до ІФНТУНГ?

Яка політика повернення в компанії X?

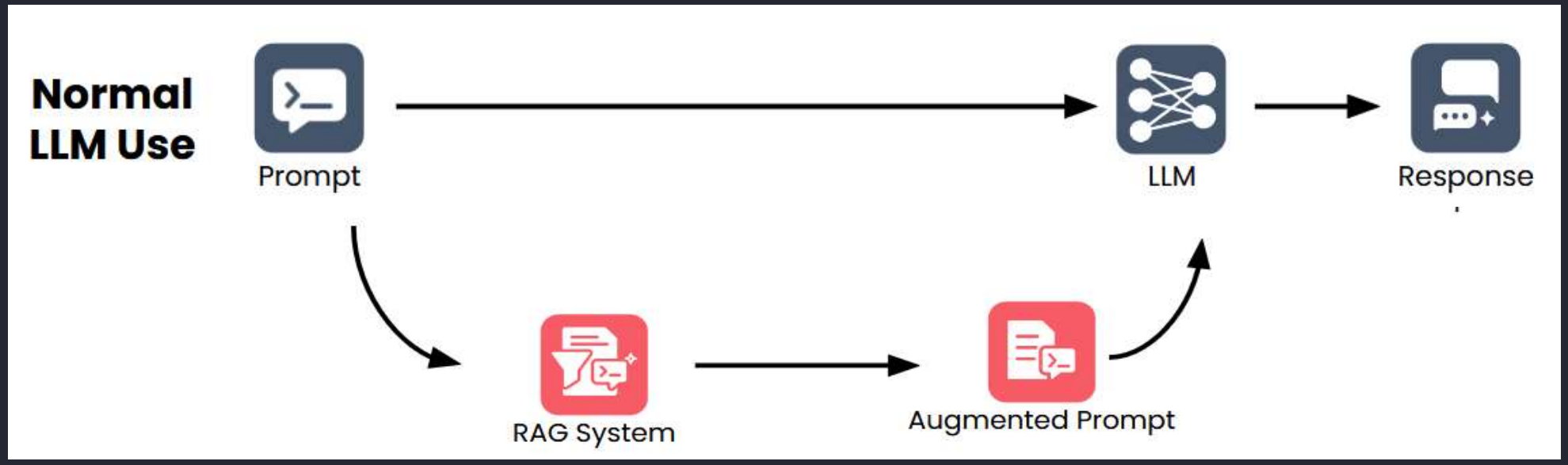
3 Дуже конкретні питання (потрібні документи / правила)

Які документи, строки подачі та вимоги для вступу на освітню програму "Комп'ютерні науки" в ІФНТУНГ у 2026 році?

Які умови повернення, строки та винятки в компанії X згідно з офіційною політикою?

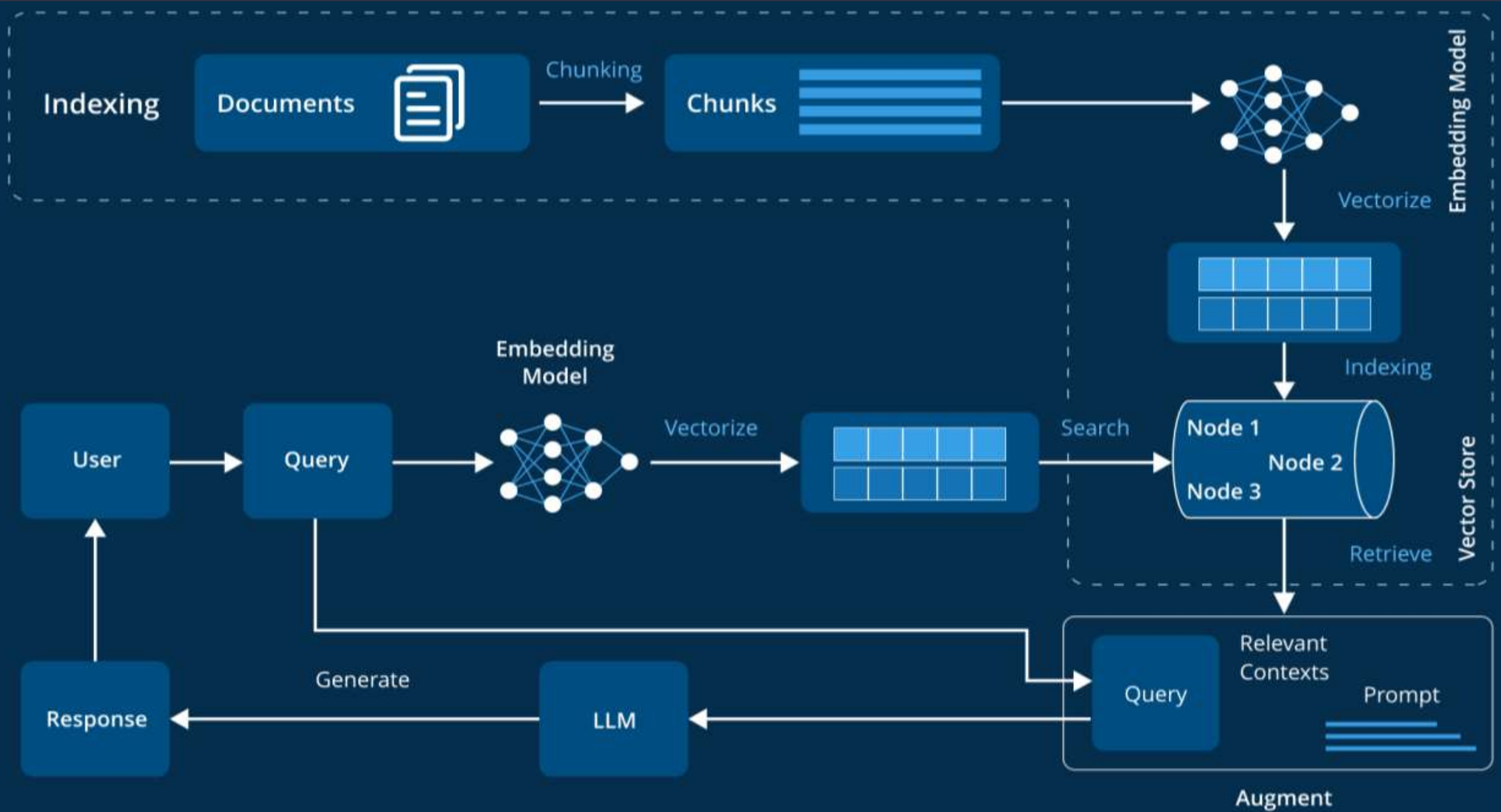
ЩО ТАКЕ RAG?

Генерація, доповнена пошуком (Retrieval Augmented Generation, RAG) - підхід, за якого модель знаходить релевантну інформацію із зовнішньої бази знань і використовує її як контекст для генерації відповіді.



Застосування: корпоративні чат-боти по внутрішній документації, технічна підтримка клієнтів, юридичні та медичні системи, навчальні асистенти

АРХІТЕКТУРА RAG-СИСТЕМИ



КОМПОНЕНТИ RAG-СИСТЕМИ

Векторна база даних

Сховище документів, статей або інших матеріалів, до якого звертається система під час обробки запиту.

Документи завантажу-ються, розбиваються на частини та зберігаються у вигляді векторних представлень.

Ретривер

Здійснює пошук релевантних фрагментів в базі знань.

При надходженні запиту система знаходить найбільш схожі частини документів із векторної бази.
Виконується при кожному запиті.

Генератор

Велика мовна модель отримує оригінальний запит та знайдені ретривером фрагменти як контекст, і формує на їх основі відповідь.

Модель не вигадує, а використовує факти для генерації відповідей.

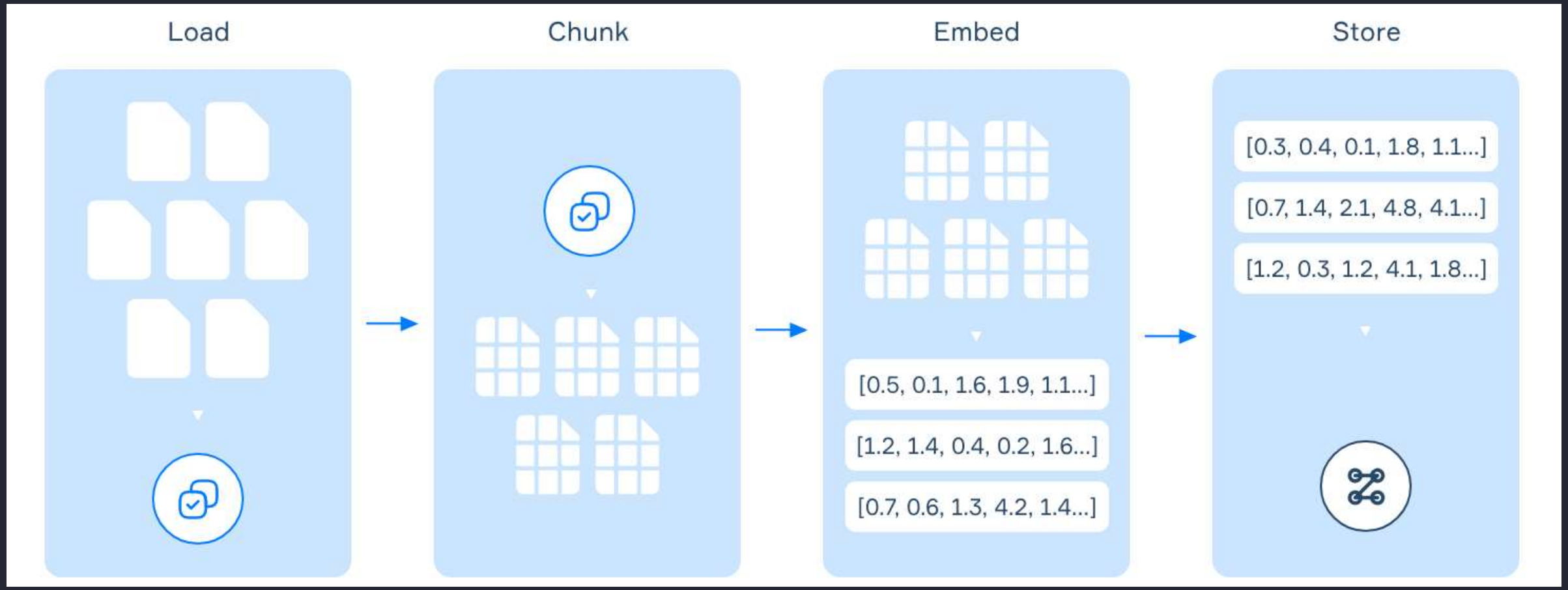
ПІДГОТОВКА БАЗИ ЗНАНЬ (INDEXING)

1 Loading - завантаження сирих даних із різних джерел (PDF, Word, веб-сторінки, бази даних)

2 Chunking - розбиття великих документів на менші фрагменти (чанки)

3 Embedding - кожен фрагмент перетворюється на вектор, який відображає його значення

4 Storing - вектори разом із оригінальним текстом зберігаються у векторній базі даних



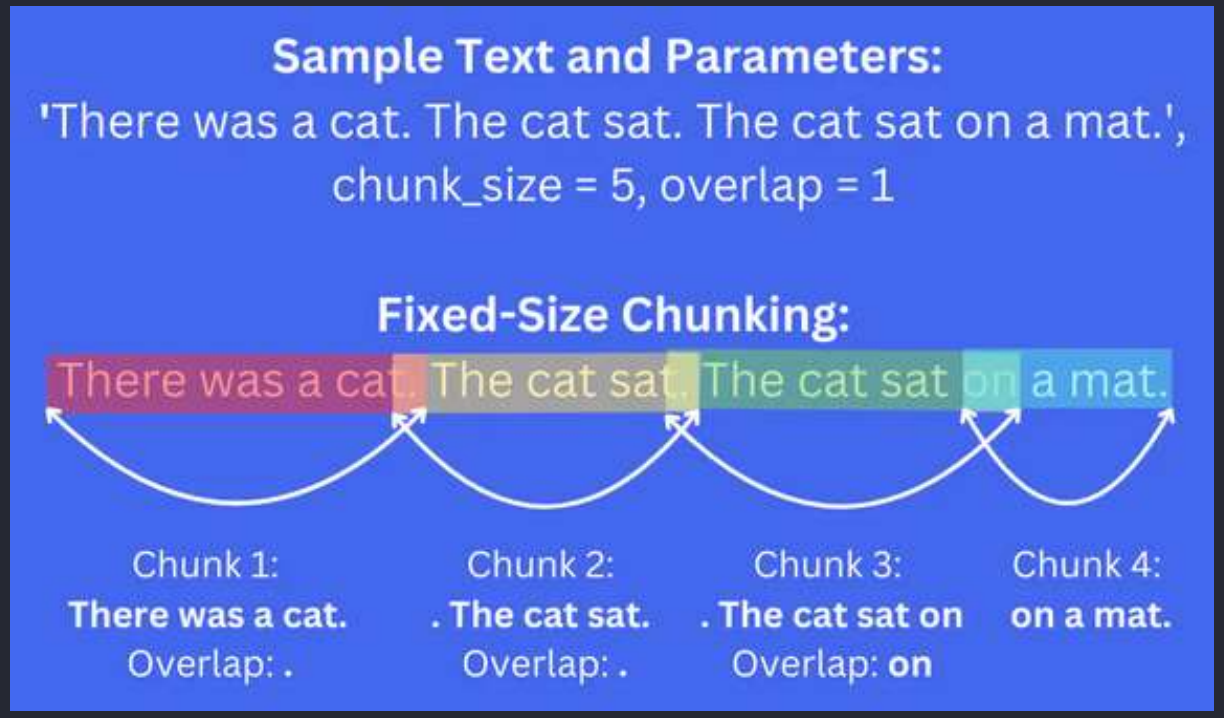
ТЕХНІКИ РОЗБИТТЯ ДОКУМЕНТІВ НА ФРАГМЕНТИ

1 Фіксований (Fixed-size) – текст ділиться на на рівні частини за кількістю символів або слів

2 Рекурсивний (Recursive) - текст послідовно ділиться за розділювачами (абзаци, рядки, речення), поки фрагменти не стануть потрібного розміру. Найпоширеніший підхід на практиці.

3 Семантичний (Semantic) - розбиття за змістом (логічно завершені фрагменти)

4 За структурою (Document-based) - враховує структуру документу (заголовки, розділи, сторінки)



EMBEDDINGS

Embeddings – це числове представлення тексту у вигляді вектора, яке відображає його зміст.

Слова з подібним контекстом мають схожі векторні представлення!

Наприклад, actor [human, profession, masculine, singular, art]

actor → [0.3, 5.2, 1, 1, -3]

actress → [0.3, 5.2, 2, 1, -3]

man → [0.1, 4.0, 1, 0, -2]

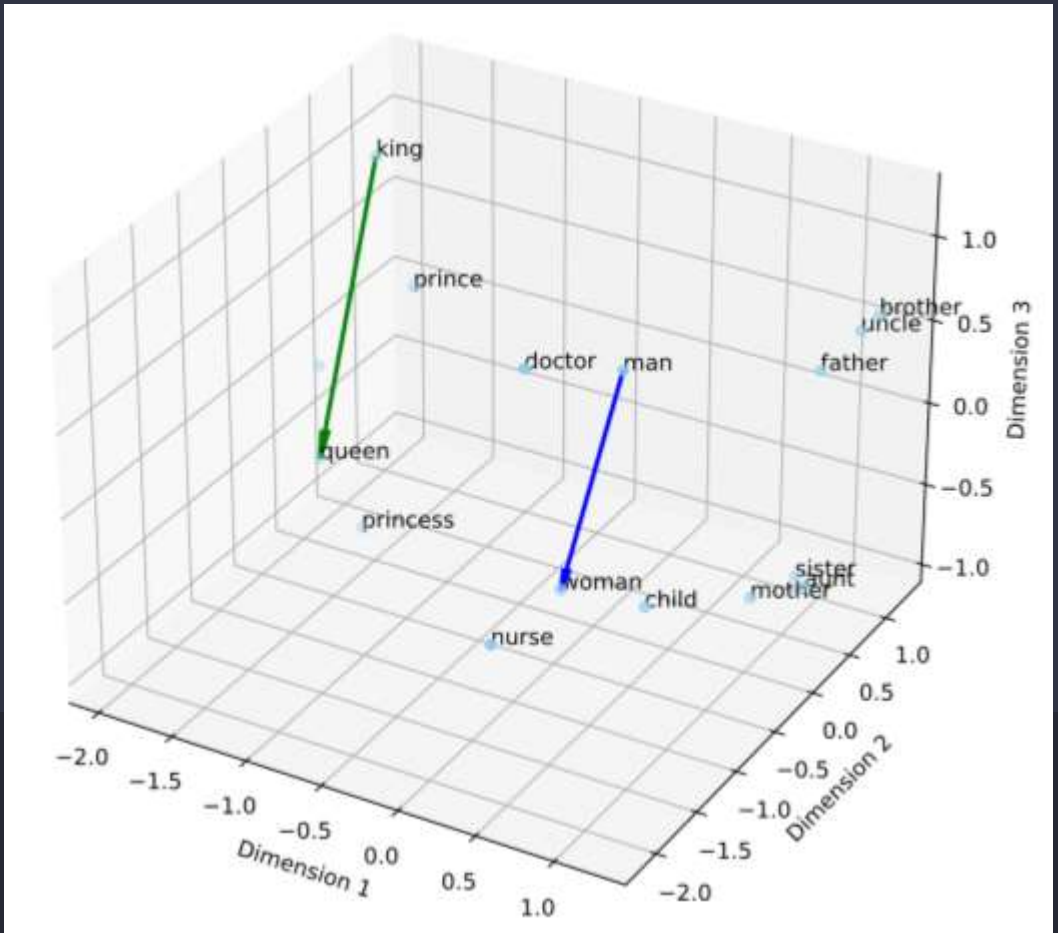
woman → [0.1, 4.0, 2, 0, -2]

actor - man + female = actress

actor - man → [0.2, 1.2, 0, 1, -1]

[0.2, 1.2, 0, 1, -1] + woman → [0.3, 5.2, 2, 1, -3]

king – man + woman = queen



ВЕКТОРНІ СХОВИЩА

Назва	Тип	Особливості	Застосування
FAISS	Локальна	Дуже швидка, працює в пам’яті, без серверу	Прототипи, дослідження, невеликі системи
Chroma	Локальна	Проста у використанні, інтеграції з LLM-фреймворками, зберігає метадані	Прототипи, навчальні проєкти
Qdrant	Серверна (open-source)	Підтримка фільтрів, API, масштабованість, можна запускати локально або в хмарі	Production-системи, складні RAG
Pinecone	Хмарна	Не потребує налаштування, висока продуктивність, автомасштабування	Комерційні продукти, швидкий старт
Weaviate	Серверна (open-source)	Вбудовані ML-модулі, гібридний пошук, підтримка мультимодальних даних	Розширені AI-системи, enterprise

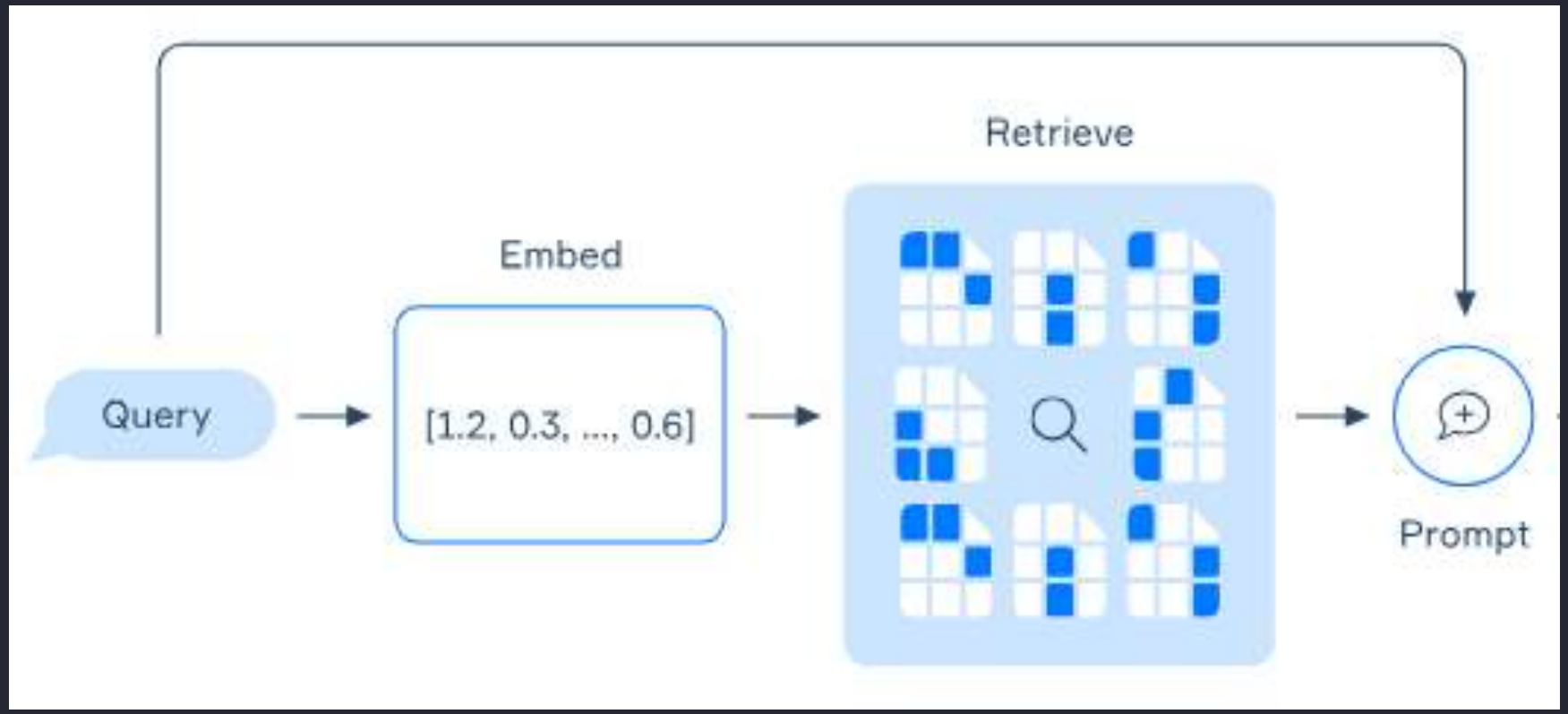
ПОШУК ПОТРІБНИХ ФРАГМЕНТІВ (RETRIEVAL)

1 Користувач вводить запит

2 Запит перетворюється на вектор (ембединг) на менші фрагменти (чанки)

3 Відбувається пошук фрагментів, вектори яких найближчі до вектора запиту.

4 Векторна база даних повертає K найближчих фрагментів

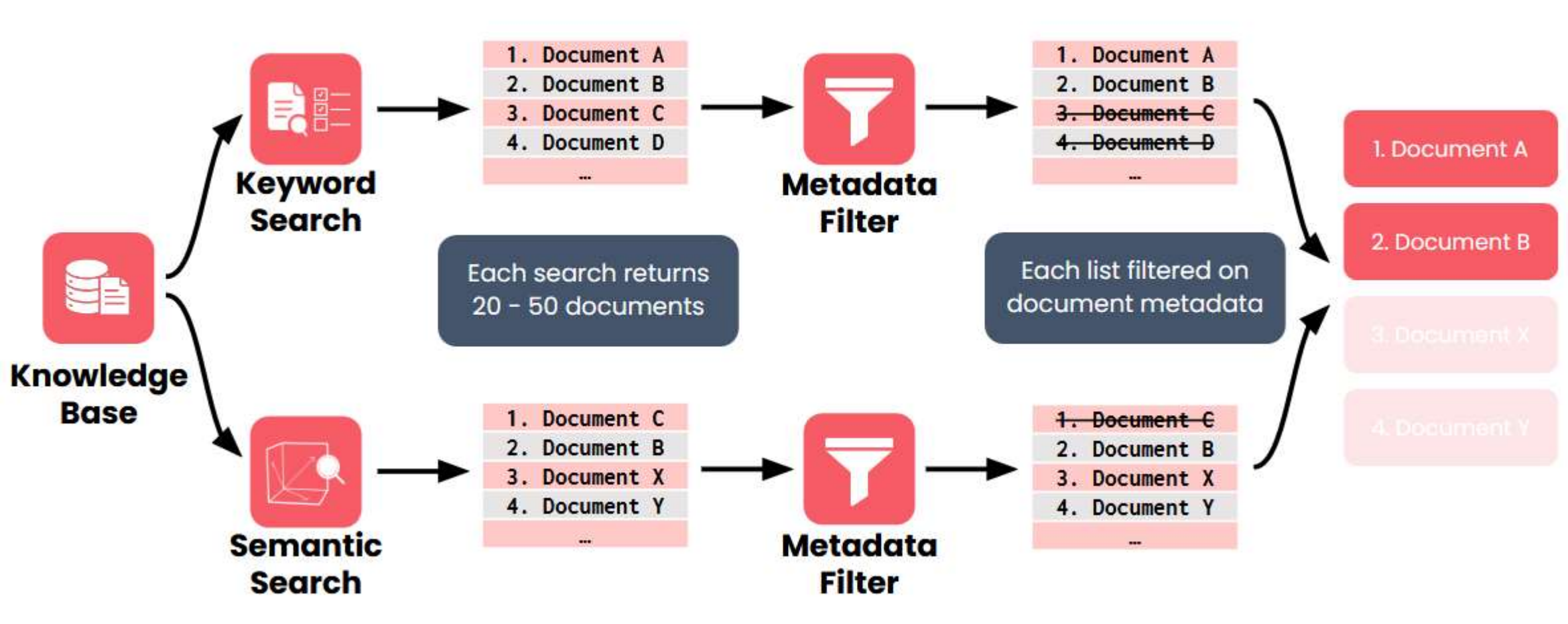


ГІБРИДНИЙ ПОШУК

1 Keyword Search. Шукає документи, що містять точні слова із запиту користувача.

2 Semantic Search. Шукає документи схожі за змістом, навіть якщо слова не збігаються.

3 Metadata Filtering. Виключає документи що не відповідають критеріям (дата, джерело, автор)

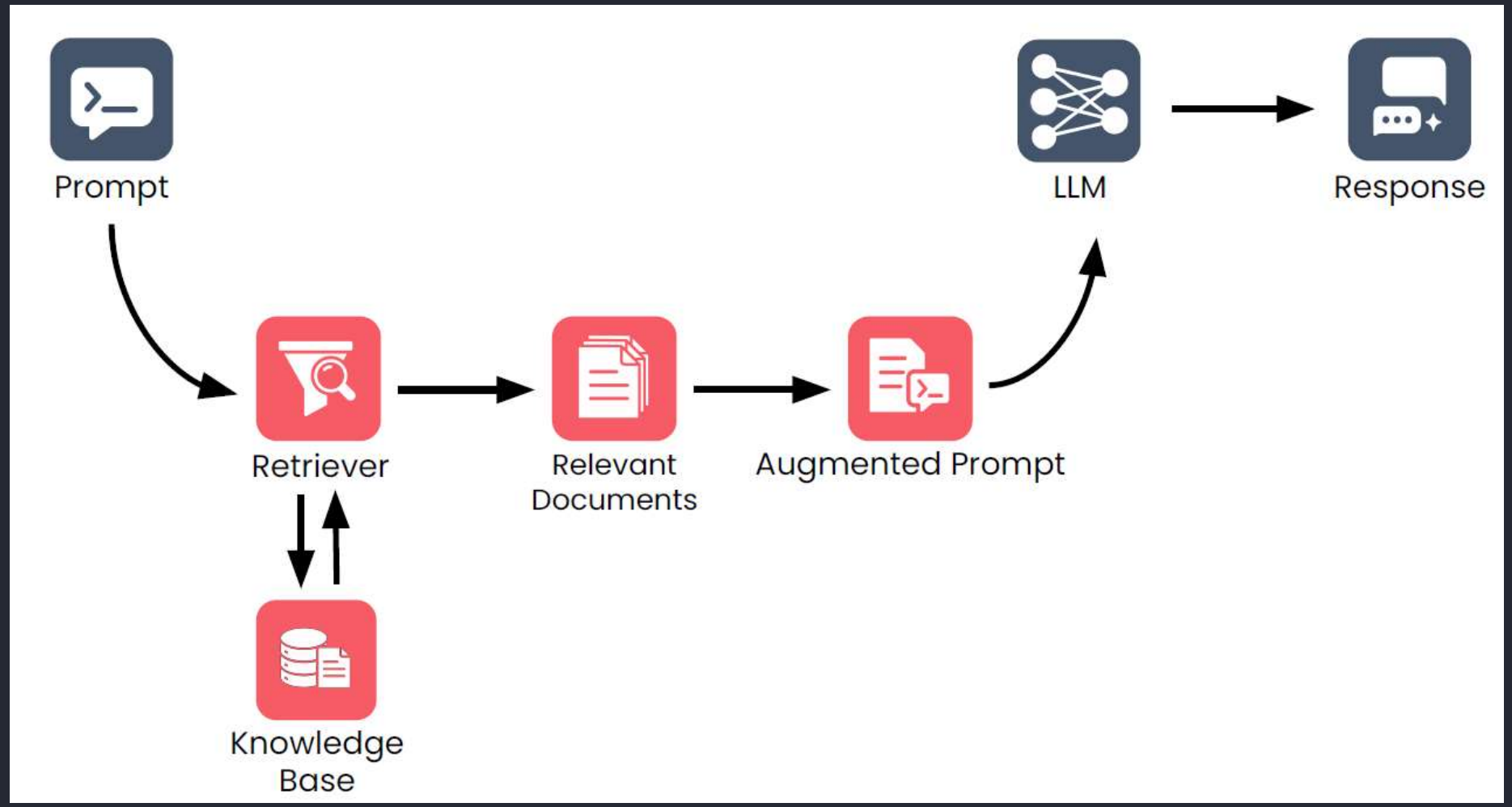


ГЕНЕРАЦІЯ ВІДПОВІДІ

Велика мовна модель (генератор) генерує відповідь, використовуючи запит і знайдений контекст.

Модель може змішувати знайдений контекст із власними знаннями, тому варто задати обмеження:

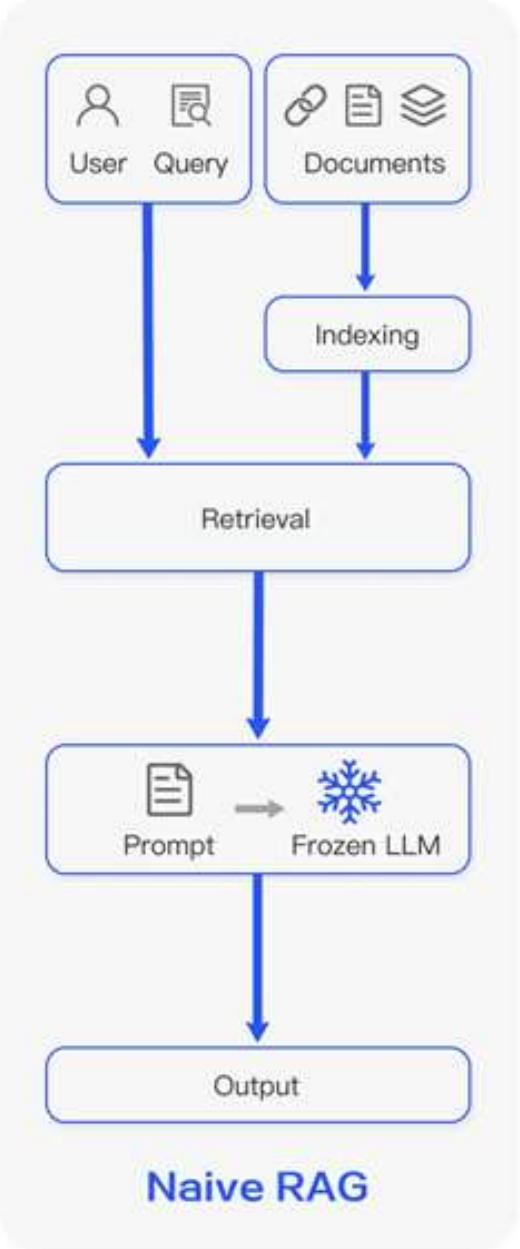
"Answer ONLY based on context"

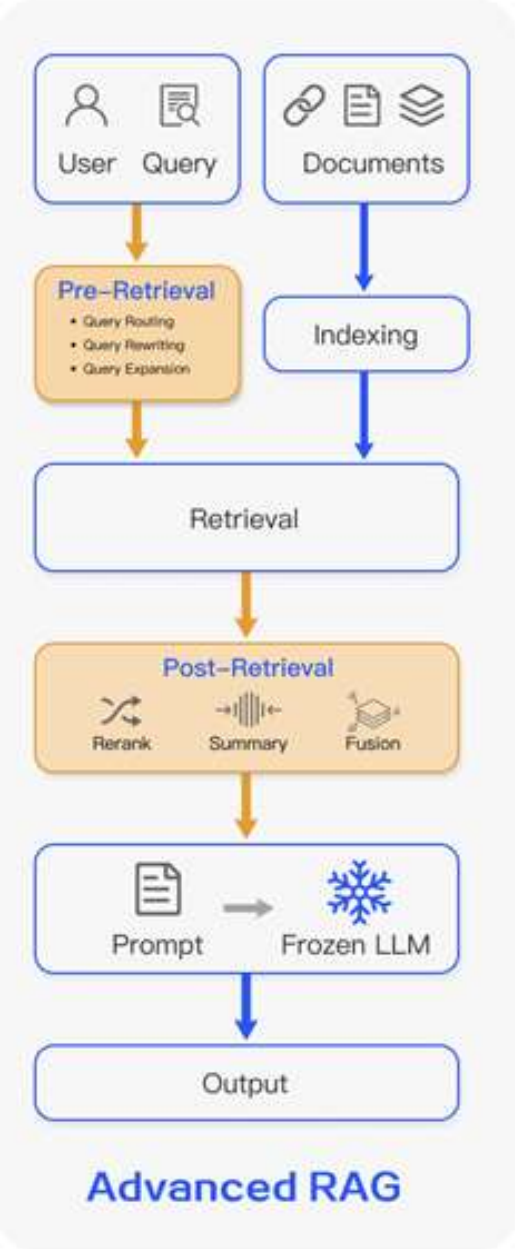


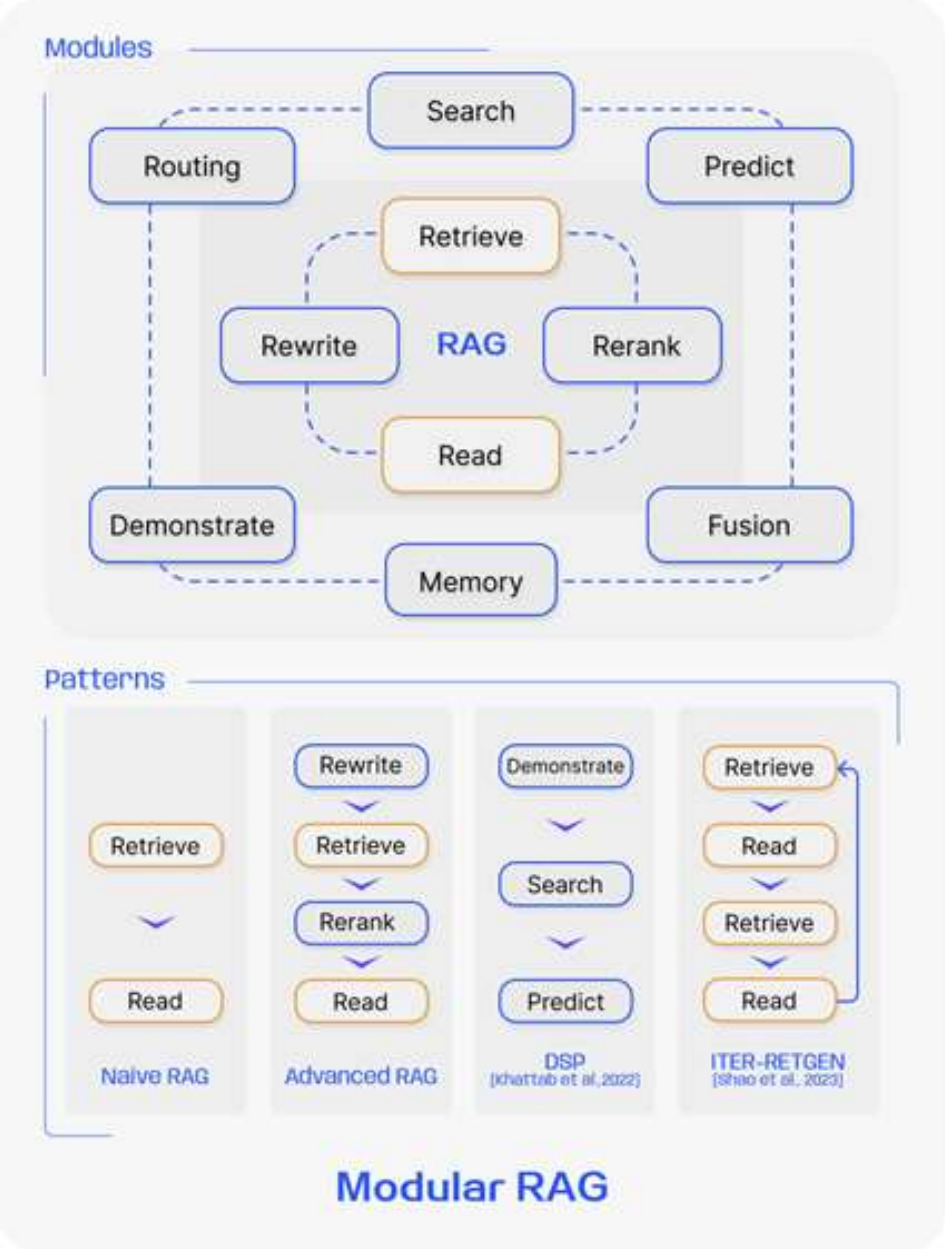
ПРИКЛАД КОДУ

```
# 1. Load documents
loader = DirectoryLoader("docs/", glob="*.pdf")
documents = loader.load()
# 2. Split documents into chunks
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
chunks = text_splitter.split_documents(documents)
# 3. Create embeddings for each chunk
embeddings = OpenAIEmbeddings(model="text-embedding-3-large")
# 4. Store embeddings in FAISS vector database
vector_store = FAISS.from_documents(documents=chunks, embedding=embeddings)
# 5. Create retriever (top-3 most similar chunks)
retriever = vector_store.as_retriever(search_type="similarity", search_kwargs={"k": 3})
# 6. Create RAG pipeline
qa_chain = RetrievalQA.from_chain_type(llm=ChatOpenAI(model="gpt-4o-mini"),
    chain_type="map_reduce", retriever=retriever)
# 7. Ask a question
response = qa_chain.invoke({"query": "Що таке RAG?"})
print(response["result"])
```

ЕВОЛЮЦІЯ RAG-СИСТЕМ







ІНСТРУМЕНТИ ДЛЯ РЕАЛІЗАЦІЇ RAG-СИСТЕМ

Категорія	Інструмент	Особливості
Frameworks	LangChain	Найбільш поширений, побудова RAG pipeline, багато інтеграцій
	LlamaIndex	Спеціалізований на RAG, орієнтований на індексацію та пошук
	LangGraph	Побудова агентських RAG-систем у вигляді графів станів
LLM	OpenAI / Anthropic / Google	Хмарні API, простий старт
	Ollama	Локальний запуск open-source моделей, безкоштовно
Vector DB	FAISS, Chroma	Локальна, швидка, проста у використанні, зручна для прототипів
Embeddings	OpenAI Embeddings	Висока якість семантичного пошуку
	SentenceTransformers	Локальні моделі, без API
Deployment	Streamlit	Найпростіший варіант, швидкий запуск, мінімум фронтенду
	Telegram Bot	Простий інтерфейс, звичний для користувачів
	FastAPI + React	Повноцінний веб-додаток, складніше реалізувати
	Railway / Render	Хмарне рішення без складної інфраструктури

З ЧОГО ПОЧАТИ БУДУВАТИ RAG-СИСТЕМУ?

RAG = Retrieval + Augmented + Generation

ДЯКУЮ ЗА УВАГУ